Streaming Spike-Token Attention for BCI Decoding

Silvia Song (jiayuns), Vikram Mani (vmani)

Project Website URL: Streaming Spike-Token Attention for BCI Decoding

Summary

We will use CUDA on a single NVIDIA GPU to study cross-attention for a streaming brain–computer interface (BCI) decoder. We will implement fresh attention and a GPU cache that reuses partial attention results across overlapping windows, then measure how the two approaches perform as spike counts and query overlap change; additionally, we will also test how assigning blocks by spike count and changing tile sizes affect per-window latency. Using those measurements, we will build and evaluate a simple policy that chooses when to reuse results and when to recompute them.

Background

A BCI decoder takes recorded brain activity and predicts something a device can use, such as how fast to move a cursor horizontally and vertically. The input to a model like POYO is a list of event tokens recording which neural unit fired and when. For each query, cross-attention compares it with every spike's key to produce scores. Softmax converts those scores into weights, and the query's output is a weighted sum of the spikes' value vectors.

The number of spikes varies across windows, while the number of queries can stay fixed. Different queries can be processed in parallel because each produces its own output. For a long window, split-key attention also lets several GPU blocks process different groups of spikes for the same query. Their partial results are merged afterward because the softmax accounts for all spikes in the window. Flash-Decoding and FlashInfer describe this decomposition and merge. FlashAttention repo (variable-length baseline; GPU support notes), Flash-Decoding for long-context inference (the split-key idea)

Moreover, successive decoding windows can share both spikes and queries. If the same query reads the same group of spikes twice, we can save its partial result and use it during the next update. The reuse count for that schedule comes from two separate overlaps:

- Queries: POYO places 32 queries at each of eight evenly spaced timestamps in the 1-second window. A 500 ms shift moves forward by four of those positions. Four groups remain, so $4 \times 32 = 128$ queries can reuse work. arxiv.org
- Spikes: For windows $[0, 1.0)$ and $[0.5, 1.5)$, count the actual spikes in $[0.5, 1.0)$. Those spikes can reuse work for the 128 retained queries, provided their effective scores, values, and masks are unchanged.

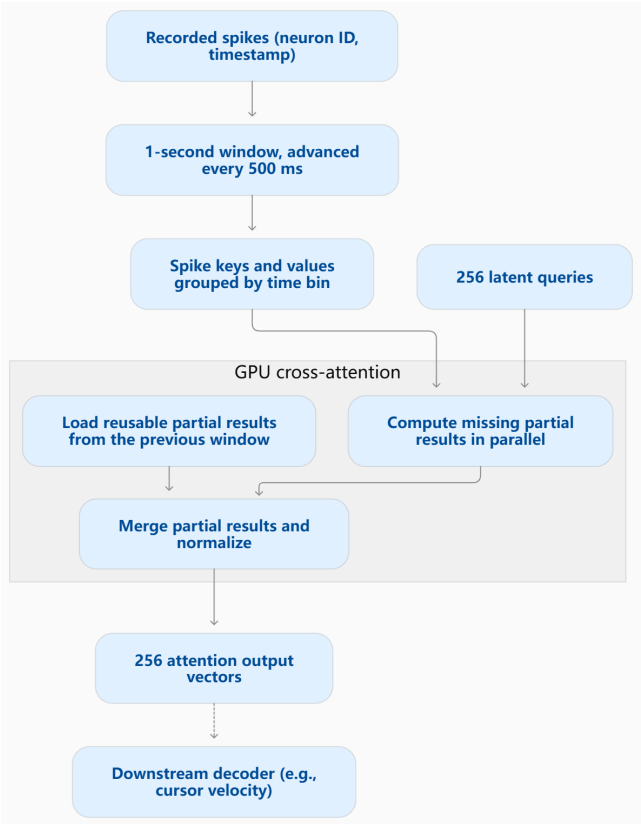


Figure 1. CUDA blocks compute partial attention for groups of queries and spikes. When the same query and spike bin carry over to the next window, the GPU can load their cached partial result and combine it with newly computed results.

The Challenge

Reducing latency through caching requires finding enough reusable work because only some query–spike pairs repeat, and reusing them adds memory reads and a merge step. When a one-second window advances by 500 ms, 128 of the 256 queries keep their time anchors. We can reuse their partial attention over the shared spikes when the scores, values, and masks stay the same. If half the spikes carry over, we skip 25% of the dot products. We want to measure when that saving outweighs the cache overhead.

Parallelizing the attention requires balancing work across blocks and combining their results because spike bins contain different numbers of events, and each query's softmax depends on all its bins. Blocks can compute dot products independently, but their partial results must be merged before the query output is ready. Giving every bin the same number of blocks can leave GPU cores idle while dense bins finish. We will assign more blocks to bins with more spikes and pack each bin's keys and values contiguously for coalesced reads. When a bin ends partway through a tile, boundary checks can leave some lanes in a warp idle. For sparse bins, each split performs fewer dot products, so writing and merging its partial result can account for a larger share of the work.

Keeping the GPU busy requires tuning tile sizes and split counts because larger tiles use more registers and shared memory, while splitting a bin across more blocks creates more partial results to merge. Larger tiles let each block reuse keys and values across more queries, but can reduce the number of blocks that fit on an SM. We will compare fresh and cached attention with the same tiling to isolate the effect of reuse, then test whether fewer splits reduce latency for sparse windows.

Resources

We will write the CUDA attention kernels from scratch and use torch_brain as a reference for spike tokenization and latent query layout: POYO and torch_brain provide a model reference and examples of how spike tokens, latent queries, and variable-length attention are handled. POYO paper, torch_brain.

Flash-Decoding and FlashInfer provide references for splitting the key/value dimension, merging partial results, and scheduling variable-length attention. Flash-Decoding, FlashInfer paper.

NLB MC_Maze_Small provides recorded spike timestamps for our replay workload.

FlashAttention-3 provides a reference for overlapping data movement with attention computation on the H100. The FlashAttention inference API shows how to update cached keys and values in place. FlashAttention-3 paper · FlashAttention repository

Goals and Deliverables

Plan to achieve:

We will implement a fresh CUDA version that computes attention from scratch for each window and a cached version that reuses partial results with load balancing and tiling. We will check both against a PyTorch reference and evaluate the following optimizations.

Cache partial attention across windows. When the one-second window advances by 500 ms, 128 of the 256 queries keep their time anchors. For those queries, we will load the cached partial result for the shared spike bin and merge it with the result for the newly arrived bin. New queries will compute attention over both bins. We will compare the fresh and cached versions with the same tiling and scheduling to isolate the effect of reuse.

Assign CUDA blocks based on spike count. Two 500 ms bins can contain very different numbers of spikes. For a given group of queries, we will split a dense bin across several blocks so they can process different spikes in parallel. A sparse bin may need only one split, avoiding extra partial results to merge. We will compare this approach with giving every bin the same number of splits.

Tune how many queries and spikes each block processes together. We will load a group of spike keys and values into shared memory so several queries can use them. We will vary the number of queries that share those inputs and the number of spikes loaded at a time. This will show whether greater data reuse offsets the extra registers or shared memory that can limit how many blocks fit on an SM.

We will report median and 99th-percentile latency per window, including cache maintenance and merging. We will use the NLB MC_Maze_Small replay to test real spike counts and synthetic windows to vary spike count and overlap independently.

Hope to achieve:

Choose when to use the cache. If we find that caching pays off above a certain amount of reusable work, we will test a policy that uses the current window's spike count and cache hits to choose between reuse and recomputation. We will compare it with always caching and always recomputing on a replay trace.

Overlap loading with attention computation. If profiling shows that blocks wait for keys and values in dense bins, we will try loading the next spike tile while computing attention on the current one. We will compare latency at the same tile size to see whether overlapping the load is worth the extra shared memory.

Reuse prepared keys and values. If we extend the benchmark to start from spike events, we will keep valid keys and values for spikes in the shared 500 ms bin and prepare vectors only for newly arriving spikes. Even new queries can use those retained vectors. We will measure the effect on latency from spike events through attention output.

If work goes more slowly. We will focus on the one-second windows with a 500 ms hop. We will validate fresh and cached attention against PyTorch and report median and 99th-percentile latency on the NLB replay using one tile configuration. We will defer the broader scheduling and tiling experiments, the adaptive cache policy, and the two FlashAttention-inspired extensions.

Platform Choice

We will use CUDA on a single NVIDIA GPU on the GHC machine. The query–spike dot products give us many independent operations, while the softmax requires us to combine results across the spike chunks. CUDA lets us control how these tasks map to thread blocks, how much state we write to memory, and how we merge it. This makes it possible to compare extra GPU parallelism, recomputation, and cache traffic on the same hardware.

Schedule

Week	Work and checkpoint

Nov. 7–15	Submit the proposal on Nov. 9. Set up the CUDA environment, prepare sample spike windows, and build the PyTorch reference.
Nov. 16–22	Implement fresh split-key attention. Check correctness and collect initial latency measurements across spike counts.
Nov. 23–29	Submit the milestone on Nov. 23 with the fresh baseline and initial measurements. Implement the cache and verify it across consecutive windows.
Nov. 30–Dec. 6	Compare fresh and cached attention on the replay workload. Test block assignment and tile sizes. If the measurements show a clear crossover, evaluate a policy for choosing when to reuse cached results.
Dec. 7–11	Finish the replay benchmarks and figures. Submit the report by Dec.10 and present the poster on Dec 11.